```
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 2
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Passed 9
Failed 1
Bonus to instructor!
```

Fig. 3.6 | Analysis of examination results. (Part 2 of 2.)

Self Check

1 *(Fill-in-the-Blank)* Control statements may be stacked on top of one another (in sequence) just as a child stacks building blocks. The only other structured way control statements may be connected in C is _____—that is, placing a control statement inside another control statement.

Answer: nesting.

3.11 Assignment Operators

C provides several assignment operators for abbreviating assignment expressions. For example, the statement

```
c = c + 3;
```

can be abbreviated with the **addition assignment operator** `+=` as

```
c += 3;
```

The `+=` operator adds the value of the expression on the operator's right to the value of the variable on the operator's left then stores the result in the variable on the left. So, the assignment `c += 3` adds `3` to `c`'s current value. The following table shows the arithmetic assignment operators, sample expressions using these operators, and explanations:

Assignment operator	Sample expression	Explanation	Assigns
Assume: `int c = 3, d = 5, e = 4, f = 6, g = 12;`			
`+=`	`c += 7`	`c = c + 7`	10 to c
`-=`	`d -= 4`	`d = d - 4`	1 to d
`*=`	`e *= 5`	`e = e * 5`	20 to e
`/=`	`f /= 3`	`f = f / 3`	2 to f
`%=`	`g %= 9`	`g = g % 9`	3 to g

Self Check

1 *(Fill-in-the-Blank)* The statement

```
b = b * 5;
```

can be abbreviated with the multiplication assignment operator *= as _____.
Answer: `b *= 5;`

2 *(Fill-in-the-Blank)* What does the assignment `c -= 3;` do? _____
Answer: Subtracts `3` from `c`'s current value.

3.12 Increment and Decrement Operators

The unary **increment operator** (`++`) and the unary **decrement operator** (`--`) add one to and subtract one from an integer variable, respectively. The following table summarizes the two versions of each operator:

Operator	Sample expression	Explanation
`++`	`++a`	Increment `a` by 1, then use the new value of `a` in the expression in which `a` resides.
`++`	`a++`	Use the current value of `a` in the expression in which `a` resides, then increment `a` by 1.
`--`	`--b`	Decrement `b` by 1, then use the new value of `b` in the expression in which `b` resides.
`--`	`b--`	Use the current value of `b` in the expression in which `b` resides, then decrement `b` by 1.

To increment the variable `c` by 1, you can use the operator `++` rather than the expressions `c = c + 1` or `c += 1`. If you place `++` or `--` before a variable (i.e., prefixed), they're referred to as the **preincrement** or **predecrement operators**. If you place `++` or `--` after a variable (i.e., postfixed), they're referred to as the **postincrement** or **postdecrement operators**. By convention, unary operators should be placed next to their operands with no intervening spaces.

Figure 3.7 demonstrates the difference between the preincrementing and the postincrementing versions of the `++` operator. Postincrementing the variable `c` causes it to be incremented *after* it's used in the `printf` statement. Preincrementing the variable `c` causes it to be incremented *before* it's used in the `printf` statement. The program displays the value of `c` before and after using `++`. The decrement operator (`--`) works similarly.

```
// fig03_07.c
// Preincrementing and postincrementing.
#include <stdio.h>

```

Fig. 3.7 | Preincrementing and postincrementing. (Part 1 of 2.)

```
// function main begins program execution
int main(void) {
   // demonstrate postincrement
   int c = 5; // assign 5 to c
   printf("%d\n", c); // print 5
   printf("%d\n", c++); // print 5 then postincrement
   printf("%d\n\n", c); // print 6

   // demonstrate preincrement
   c = 5; // assign 5 to c
   printf("%d\n", c); // print 5
   printf("%d\n", ++c); // preincrement then print 6
   printf("%d\n", c); // print 6
} // end function main
```

```
5
5
6

5
6
6
```

Fig. 3.7 | Preincrementing and postincrementing. (Part 2 of 2.)

The three assignment statements in Fig. 3.6

```
passes = passes + 1;
failures = failures + 1;
student = student + 1;
```

can be written more concisely with assignment operators as

```
passes += 1;
failures += 1;
student += 1;
```

with preincrement operators as

```
++passes;
++failures;
++student;
```

or with postincrement operators as

```
passes++;
failures++;
student++;
```

When incrementing or decrementing a variable in a statement by itself, the preincrement and postincrement forms have the same effect. It's only when a variable appears in the context of a larger expression that preincrementing and postincrementing have different effects (and similarly for predecrementing and postdecrementing).

Only a simple variable name may be used as a `++` or `--` operator's operand. Attempting to use the increment or decrement operator on an expression other than a simple variable name is a syntax error—e.g., `++(x + 1)`.

ERR

C generally does not specify the order in which an operator's operands will evaluate. We'll see exceptions to this for a few operators in the next chapter. To avoid subtle errors, the ++ and -- operators should be used only in statements that modify exactly one variable.

The following table lists in decreasing precedence order the operators shown so far.

Operators	Grouping	Type
++ *(postfix)* -- *(postfix)*	right to left	postfix
+ - (*type*) ++ *(prefix)* -- *(prefix)*	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment

The third column names the various groups of operators. Notice that the conditional operator (?:), the unary operators increment (++), decrement (--), plus (+), minus (-) and casts, and the assignment operators =, +=, -=, *=, /= and %= group right-to-left. The other operators group left-to-right.

Self Check

1 ***(Multiple Choice)*** Given the following code:

```
--i;
```

which of the following statements describes what this code does?

a) Increment i by 1, then use the new value of i in the expression in which i resides.
b) Use i's current value in the expression in which i resides, then increment i by 1.
c) Decrement i by 1, then use the new value of i in the expression in which i resides.
d) Use i's current value in the expression in which i resides, then decrement i by 1.

Answer: c.

2 ***(What Does This Code Do?)*** What does this program display as x's final value?

```
#include <stdio.h>

int main(void) {
   int x = 7;
   printf("%d\n", x);
   printf("%d\n", x++);
   printf("%d\n\n", x);
```

```
    x = 8;
    printf("%d\n", x);
    printf("%d\n", ++x);
    printf("%d\n", x);
}
```

Answer: 9.

3 *(Multiple Choice)* Which of the following expressions contains a syntax error?

a) `++x + 1`
b) `x++ + x`
c) `++(x) + 1`
d) `++(x + 1)`

Answer: d. Only a simple variable name may be used as the operand of a `++` or `--` operator. Attempting to use the increment or decrement operator on an expression other than a simple variable name is a syntax error—e.g., `++(x + 1)`.

SEC 3.13 Secure C Programming

Arithmetic Overflow

Figure 2.4 presented an addition program that calculated the sum of two `int` values with the statement

```
sum = integer1 + integer2; // assign total to sum
```

ERR Even this simple statement has a potential problem. Adding the integers could result in a value too large to store in the `int` variable `sum`. This is known as **arithmetic overflow** and can cause undefined behavior, possibly leaving a system open to attack.

The constants `INT_MAX` and `INT_MIN` represent the platform-specific maximum and minimum values that can be stored in an `int` variable. These constants are defined in the header `<limits.h>`. There are similar constants for the other integral types that we'll introduce in the next chapter. You can see your platform's values for these constants by opening the header `<limits.h>` in a text editor.[2]

It's good practice to ensure that before you perform arithmetic calculations like the one above, they will not overflow. For an example, see the CERT website.

```
https://wiki.sei.cmu.edu/confluence/display/c/
```

Search for guideline INT32-C. The code uses the `&&` (logical AND) and `||` (logical OR) operators, which we discuss in Chapter 4. In industrial-strength code, you should perform checks like these for all calculations. Later chapters show other programming techniques for handling such errors.

`scanf_s` and `printf_s`

The C11 standard's Annex K introduced more secure versions of `printf` and `scanf` called `printf_s` and `scanf_s`. We discuss these functions and the corresponding secu-

2. Use your system's search feature to locate the file `limits.h`.

rity issues in Sections 6.13 and 7.13. Annex K is designated as optional, so not every C vendor implements it. In particular, the GNU C++ and Clang C++ compilers do not implement Annex K, so using `scanf_s` and `printf_s` might compromise your code's portability among compilers.

Microsoft implemented its own Visual C++ versions of `printf_s` and `scanf_s` before the C11 standard, and its compiler immediately began issuing warnings for every `scanf` call. The warnings said that `scanf` was deprecated—it should no longer be used—and that you should consider using `scanf_s` instead. Microsoft now treats what used to be a warning about `scanf` as an error. A program with `scanf` will not compile on Visual C++ and you will not be able to execute the program.

Many organizations have coding standards that require code to compile without warning messages. There are two ways to eliminate Visual C++'s `scanf` warnings—use `scanf_s` instead of `scanf` or disable these warnings. For the input statements we've used so far, Visual C++ users can simply replace `scanf` with `scanf_s`. You can disable the warning messages in Visual C++ as follows:

1. Type *Alt F7* to display the **Property Pages** dialog for your project.
2. In the left column, expand **Configuration Properties > C/C++** and select **Preprocessor**.
3. In the right column, at the end of the value for **Preprocessor Definitions**, insert

   ```
   ;_CRT_SECURE_NO_WARNINGS
   ```

4. Click **OK** to save the changes.

You'll no longer receive warnings on `scanf` (or any other functions that Microsoft has deprecated for similar reasons). For industrial-strength coding, disabling the warnings is discouraged. We'll say more about `scanf_s` and `printf_s` in a later Secure C Coding Guidelines section.

Self Check

1 *(Fill-in-the-Blank)* Adding two integers such that the result is a value that's too large to store in an `int` variable is known as arithmetic _____ and can cause undefined behavior, possibly leaving a system open to attack.
Answer: overflow.

2 *(Fill-in-the-Blank)* The platform-specific maximum and minimum values that can be stored in an `int` variable are represented by the constants `INT_MAX` and `INT_MIN`, respectively, which are defined in the header _____.
Answer: `<limits.h>`.

Summary

Section 3.1 Introduction

- Before writing a program to solve a particular problem, you must have a thorough understanding of the problem and a carefully planned approach to solving it.